

Homework Tracker

Cloud-Native DevOps Capstone Project

KUBERNETES CI/CD TERRAFORM GITOPS DEVSECOPS

Krish Rabari · DevOps Heroes Advanced Curriculum

📋 Agenda

- → Project Overview & Motivation
- → System Architecture
- → Technology Stack
- → CI/CD & DevSecOps Pipeline
- → Kubernetes & Helm Deployment
- → GitOps with Argo CD
- → Infrastructure as Code (Terraform)
- → Observability & Autoscaling
- → Security Governance
- → Live Demo & Q&A

📌 Project Overview

An **enterprise-grade, cloud-native** application to track, schedule, and manage course assignments — built to demonstrate **full-lifecycle DevOps engineering**.

3	10	6	100%
Microservices	DevOps Milestones	Security Gates	IaC Managed

React 18 Frontend · FastAPI Backend · PostgreSQL 16 Database

🏛️ System Architecture

🖥️ Frontend Tier

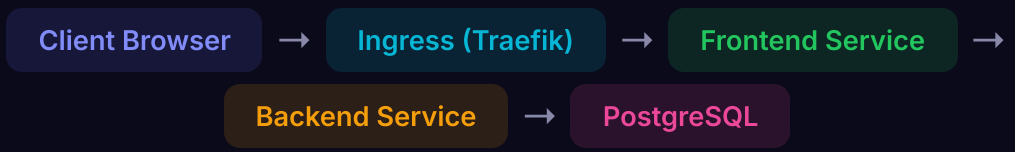
- → React 18 + Vite SPA
- → Nginx Alpine serving
- → 2 replicas (load balanced)

⚡ Backend API Tier

- → FastAPI (Python 3.12)
- → Async REST endpoints
- → HPA: 2-5 replicas
- → Non-root container (UID 10001)

🗄️ Data Persistence

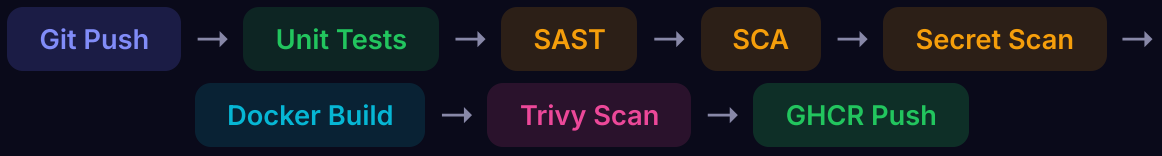
- → PostgreSQL 16
- → SQLAlchemy ORM
- → Alembic migrations
- → Persistent Volume Claims



🛠️ Technology Stack

Domain	Technology	Purpose
Frontend	React 18, Vite, Nginx	Modern responsive SPA dashboard
Backend API	Python 3.12, FastAPI	Asynchronous RESTful API
Database	PostgreSQL 16, Alembic	Relational storage, migrations
Containers	Docker, Compose v2	Multi-stage, non-root builds
Orchestration	K3s, Kind, Kubernetes	Self-healing, service discovery
Packaging	Helm 3	Templated manifests, rollbacks
CI/CD	GitHub Actions	Automated pipeline, artifact publishing
Security	Bandit, Trivy, pip-audit	SAST, SCA, container CVE scanning
GitOps	Argo CD	Declarative continuous reconciliation
IaC	Terraform (AzureRM)	Cloud provisioning & teardown
Observability	Prometheus, Grafana	Metrics, dashboards, alerting

🚀 CI/CD & DevSecOps Pipeline



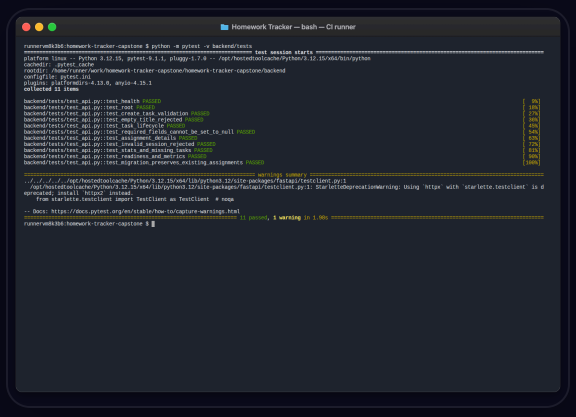
🧪 Continuous Integration

- → 11 Pytest unit & API tests
- → Bandit SAST (Medium/High severity)
- → pip-audit dependency analysis
- → Trivy filesystem secret scanning
- → Multi-stage Docker image build
- → Container image CVE scanning

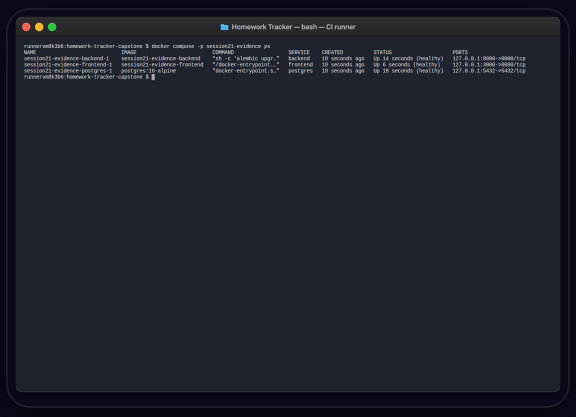
📦 Continuous Delivery

- → Publish to GitHub Container Registry
- → Disposable Kind cluster validation
- → Helm chart lint & test deploy
- → Automated HPA load testing
- → Argo CD GitOps sync verification
- → Zero critical/high CVE gate policy

🧪 Testing & Quality Evidence



Pytest — All 11 tests passing



Docker Compose — All services healthy

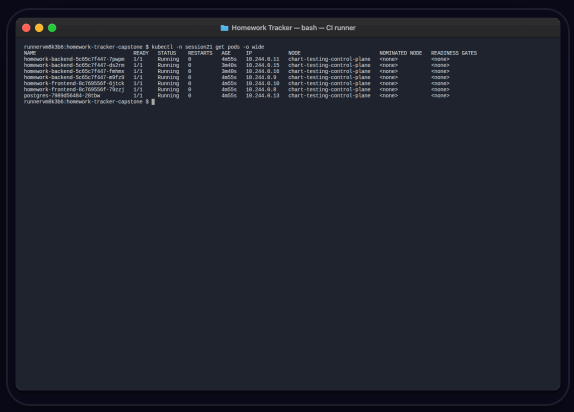
☸ Kubernetes & Helm Deployment

Helm Chart Features

- → Fully parameterized values
- → Configurable replicas & resources
- → Dynamic PVC storage provisioning
- → HPA (CPU ≥ 50%, 2-5 replicas)
- → Startup, readiness, liveness probes
- → Traefik Ingress routing

Cluster Environments

- → **Kind** — CI validation cluster
- → **K3s on Azure** — Production cloud

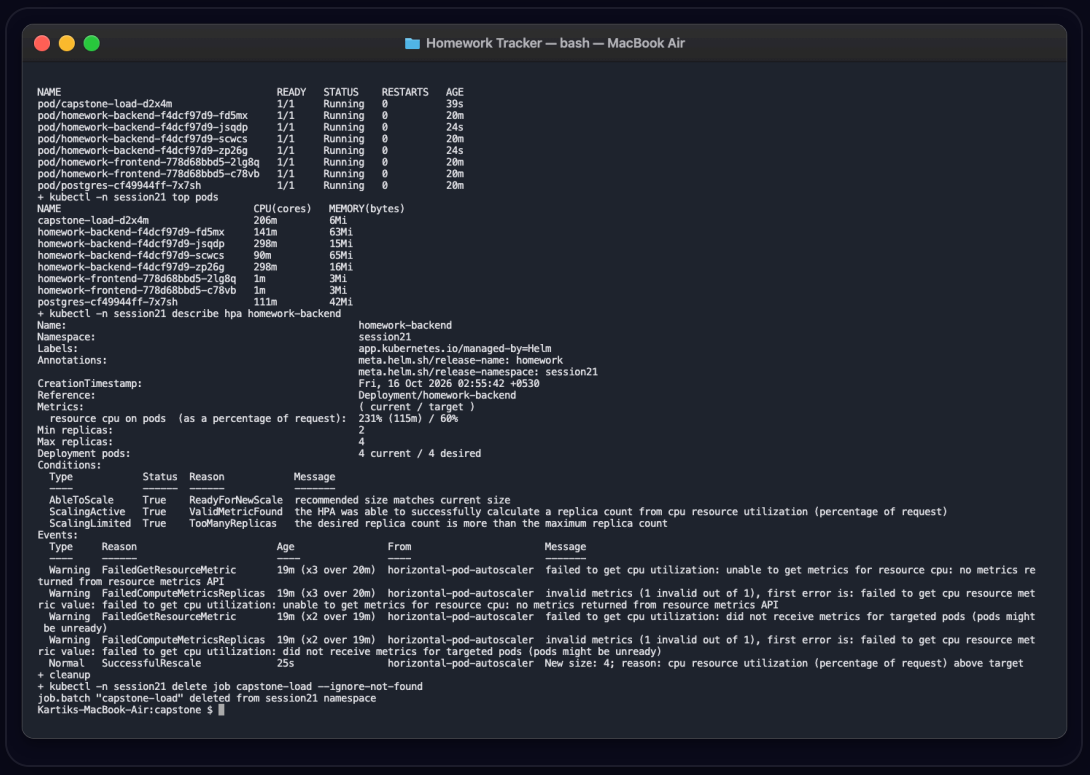


Running Kubernetes Pods



Helm release verification

📈 Horizontal Pod Autoscaling



CPU-based autoscaling under load on Azure K3s

Metric

CPU Utilization ≥ 50%

Replica Range

Min: 2 → Max: 5

Stabilization

30-second cooldown window

🔄 GitOps with Argo CD

Declarative Delivery

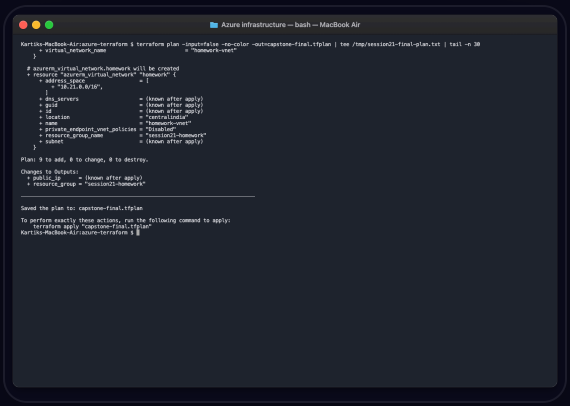
- → Git as single source of truth
- → Automated self-healing
- → Zero configuration drift
- → Continuous reconciliation loop
- → AppProject resource isolation
- → Verified in CI pipeline



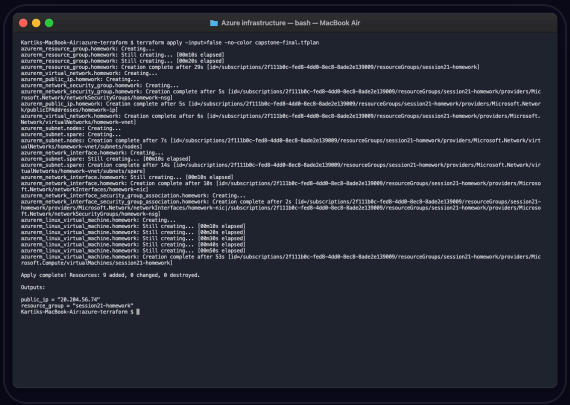
Argo CD reconciliation status

Any manual change to the cluster is **automatically reverted** to match the declared Git state.

☁️ Infrastructure as Code — Terraform



Terraform plan output



Terraform apply — resources provisioned

Azure Resources Provisioned

- → Resource Group (rg-homework-tracker)
- → Virtual Network (10.21.0.0/16)
- → Subnet (10.21.1.0/24)
- → Network Security Group (SSH + HTTP/S)
- → Ubuntu 24.04 LTS VM with K3s
- → Cloud-init: K3s, Helm, Prometheus, Traefik

Full Lifecycle

- → `terraform plan` → Preview changes
- → `terraform apply` → Provision
- → `terraform destroy` → Clean teardown

🔒 DevSecOps & Security Governance

Zero tolerance for high/critical vulnerabilities before any artifact is published.

Security Domain	Tool	Policy	Action on Failure
SAST	Bandit	Medium / High severity	Fails CI pipeline
SCA	pip-audit & npm audit	CVSS ≥ 7.0	Fails CI pipeline
SECRETS	Trivy FS Scanner	Any detected secret	Fails CI pipeline
CONTAINERS	Trivy Image Scanner	HIGH / CRITICAL CVEs	Blocks GHCR publish
RUNTIME	Non-root execution	UID 10001 / non-root	Enforced in Dockerfile

📊 Observability Stack

Prometheus Metrics

- → CPU & memory utilization per container
- → HTTP request rates & latency (p50, p95, p99)
- → Database connection pool status
- → Custom /metrics endpoint

Grafana Dashboards

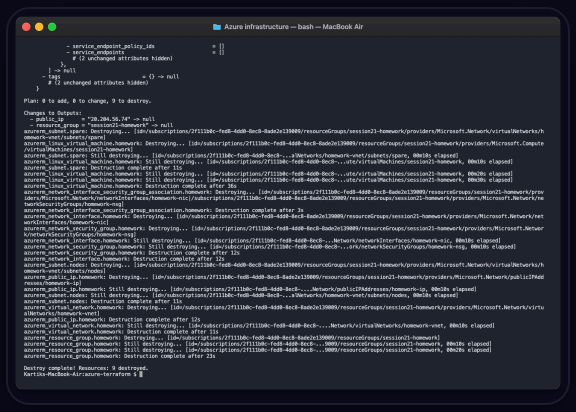
- → Real-time cluster health visualization
- → Pod-level resource consumption
- → Alert rules for threshold breaches
- → Custom dashboard configurations

Health Endpoints

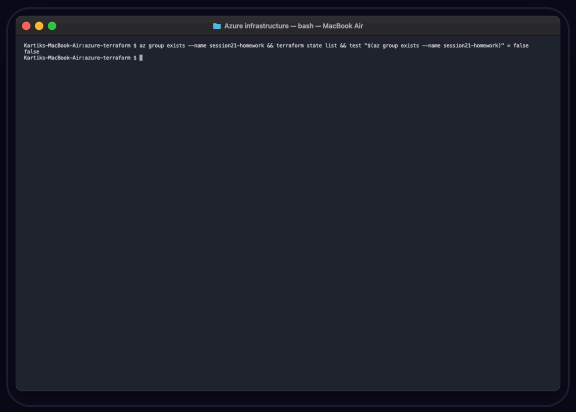
`/health`	Container liveness
`/ready`	DB readiness check
`/metrics`	Prometheus scraping

Kubernetes probes (startup, readiness, liveness) ensure **zero-downtime deployments** and automatic unhealthy pod replacement.

🧹 Clean Teardown & Cost Management



Terraform destroy — all resources removed



Azure portal — resource group deleted

Lifecycle Management

- → Single-command infrastructure teardown
- → No orphaned Azure resources
- → Cost-conscious ephemeral deployments
- → Reproducible provision → demo → destroy cycle

Full cloud lifecycle verified:

`plan` → `apply` → `demo` → `destroy`

🎬 Live Demo Walkthrough

Recorded demonstration — `capstone-walkthrough.mp4` (5 min 40 sec)

Time	Demonstration
00:00	Azure infrastructure provision (Terraform)
00:52	Kubernetes cluster & initial Helm release
01:37	Create assignment, verify persistence
01:43	Commit change & publish via CI pipeline
01:57	Tests, Docker Compose, API validation
02:34	CPU load test & HPA autoscaling
03:21	Broken image & service recovery
03:37	Prometheus & Grafana on Azure
04:09	Successful standalone CI pipeline
05:29	Updated frontend with existing data
05:34	Terraform destroy — cleanup

🏆 Key Achievements

✅ Full DevOps Lifecycle

End-to-end automation from code commit to production deployment with zero manual intervention.

✅ Self-Healing GitOps

Argo CD continuously reconciles cluster state with zero configuration drift.

✅ Security-First Approach

6 independent security gates block any release with critical vulnerabilities.

✅ Elastic Scaling

HPA-driven autoscaling responds to real load, verified with automated stress tests.

✅ Production-Ready Infrastructure

IaC-managed Azure environment with automated K3s bootstrap via cloud-init.

✅ Complete Observability

Prometheus metrics, Grafana dashboards, alerting rules, and structured health probes.



Thank You!

Questions & Discussion

Krish Rabari

GitHub: @Rabari9999

rabarikrishramjibhai2905@gmail.com

Repository: github.com/Rabari9999/homework-tracker-capstone